



Infraestructura en redes y cyber seguridad

Seguridad en Redes I

Examen

Gary Avila

LN4-B

Objetivo:

Implementar una infraestructura de red segmentada mediante VLANs en el emulador GNS3, configurando un enrutamiento perimetral del tipo *Router-on-a-Stick*, con el fin de auditar, interceptar y alertar en tiempo real el tráfico inter-VLAN (ICMP, HTTP, SSH y Escaneos Nmap XMAS) mediante un Sistema de Detección de Intrusos (NIDS) Snort desplegado en una máquina virtual de Ubuntu Server.

Objetivos Específicos

Resolver las limitaciones de hardware y almacenamiento en el entorno virtual, optimizando la topología mediante la migración del enrutador a una imagen de Cisco c3725 ligera basada en Dynamips.

Configurar subinterfaces lógicas en el enrutador aplicando la encapsulación bajo el estándar IEEE 802.1Q (*dot1Q*) para permitir la comunicación controlada entre segmentos de red independientes.

Enlazar el sistema operativo Ubuntu Server a la infraestructura simulada, habilitando el modo promiscuo en la interfaz para permitir que el motor de firmas de Snort analice profundamente los paquetes de auditoría.

Introducción

La segmentación lógica de redes utilizando Redes de Área Local Virtuales (VLANs) es una de las mejores prácticas de ciberseguridad para mitigar los dominios de difusión (*broadcast*) y aislar entornos

corporativos críticos. Sin embargo, para que los datos viajen entre distintas VLANs se requiere de un dispositivo de Capa 3 del modelo OSI que realice las funciones de enrutamiento. La técnica tradicional para lograr esto optimizando hardware es el *Router-on-a-Stick*, la cual eleva todo el tráfico a un enrutador central a través de un único enlace troncal físico compartido.

A la par de esta arquitectura, la visibilidad del tráfico se vuelve crucial. Un Sistema de Detección de Intrusos basado en Red (NIDS) como Snort complementa la topología actuando como un sensor de escucha pasiva. Al configurar el adaptador de red en modo promiscuo y acoplarlo al switch troncal, Snort analiza las cabeceras de los protocolos de transporte y red (Capa 3 y Capa 4), contrastándolas con un motor de firmas locales para identificar actividades de reconocimiento táctico y accesos no autorizados antes de que penetren los endpoints.

Desarrollo:

Fase 1: Optimización de Hardware y Despliegue en GNS3

Inicialmente, se intentó implementar un router virtualizado Huawei AR1000v. Sin embargo, debido a restricciones de almacenamiento en el disco anfitrión (con capacidad disponible por debajo del 3.10%) y la alta demanda de memoria del nodo (4096 MiB de RAM), el entorno experimentó bloqueos.

Como solución de ingeniería, se purgaron más de 20,000 archivos temporales corruptos del sistema de intercambio de disco, se incrementó la RAM de la GNS3 VM a 6-8 GB y se importó un Router Cisco c3725 a través del motor Dynamips. Este nodo se configuró de manera óptima con solo 128 MiB

(Mebibytes) de RAM y se calculó su valor Idle-PC (0x60c09aa0) en color verde para estabilizar el uso del procesador de la máquina física de Windows.

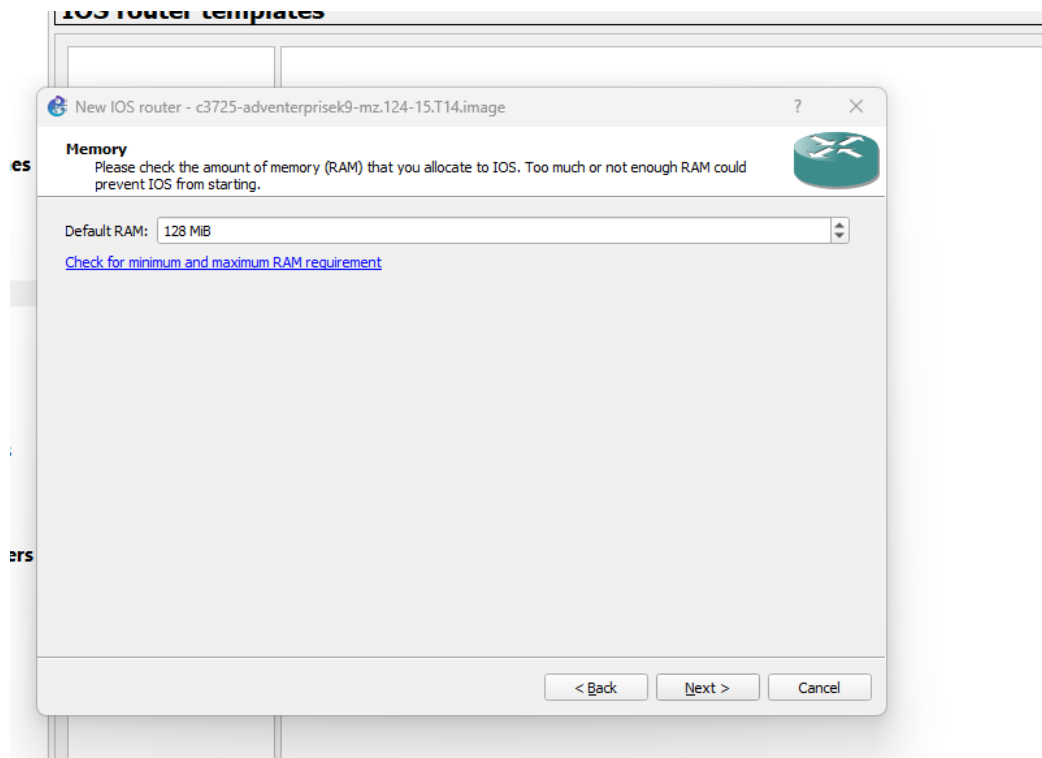


Ilustración 1 optimización de RAM del equipo Cisco

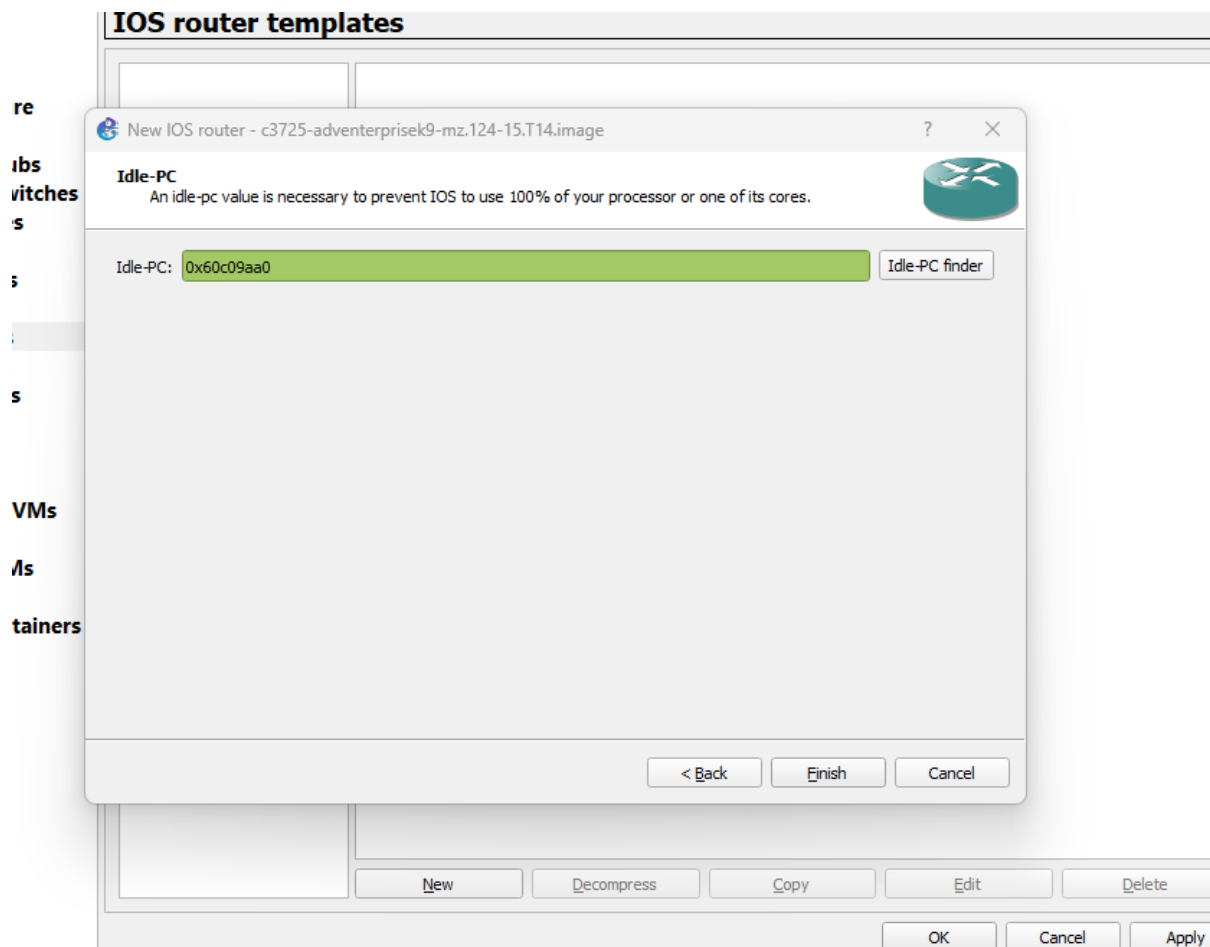


Ilustración 2 calculo Idle-PC del router

Fase 2: Configuración del Enrutamiento Inter-VLAN (Router-on-a-Stick)

El switch integrado de la topología trabaja exclusivamente en la Capa 2 (Enlace de Datos) procesando direcciones MAC y celdas lógicas de VLAN, por lo que carece de capacidades para comunicar redes IP distintas de forma directa. Para suplir esta carencia, se conectó un único cable desde la interfaz física FastEthernet0/0 del Cisco R1 hacia el puerto troncal del switch.

En el router, se segmentó la interfaz física en subinterfaces lógicas y se aplicó el estándar de etiquetado IEEE 802.1Q (dot1Q) para clasificar los paquetes procedentes del switch:

R1# configure terminal

```
R1(config)# interface FastEthernet 0/0.10
```

```
R1(config-subif)# encapsulation dot1Q 10
```

```
R1(config-subif)# ip address 192.168.10.1 255.255.255.0
```

```
R1(config-subif)# exit
```

```
R1(config)# interface FastEthernet 0/0.20
```

```
R1(config-subif)# encapsulation dot1Q 20
```

```
R1(config-subif)# ip address 192.168.20.1 255.255.255.0
```

```
R1(config-subif)# exit
```

```
R1(config)# interface FastEthernet 0/0.30
```

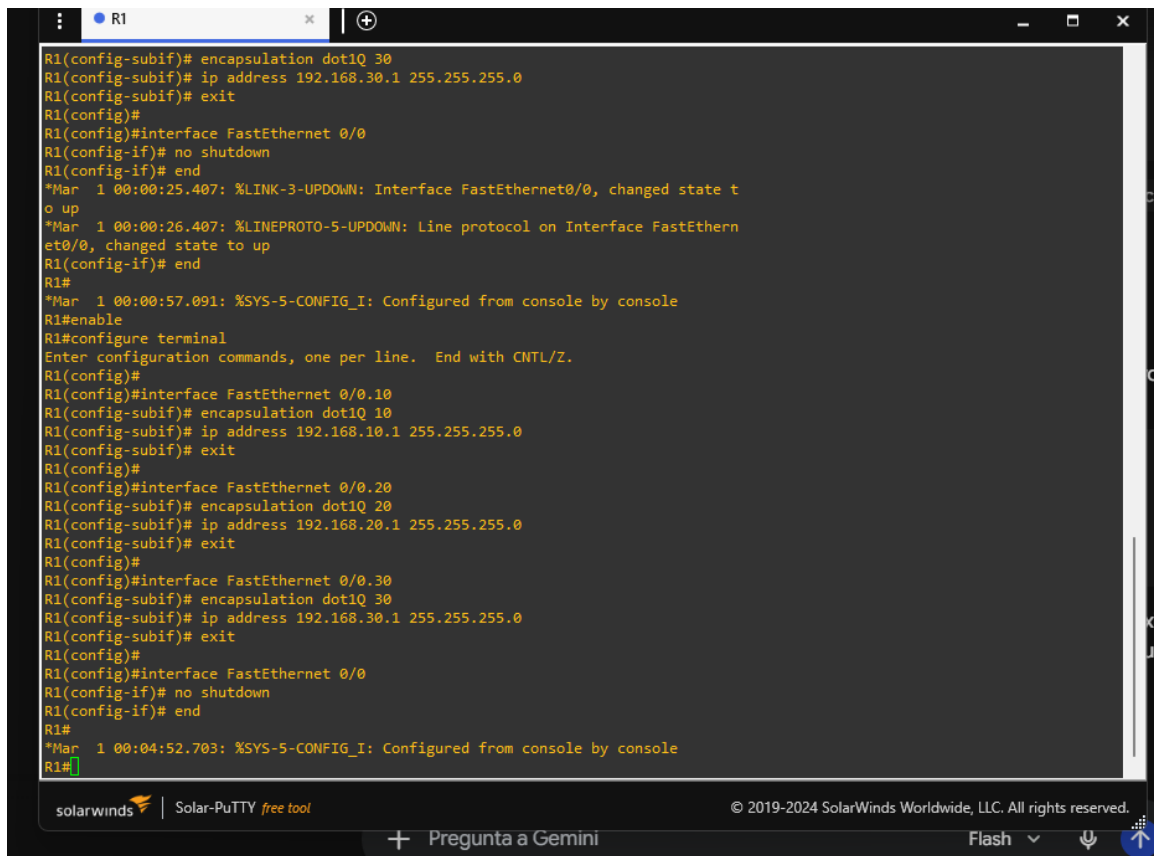
```
R1(config-subif)# encapsulation dot1Q 30
```

```
R1(config-subif)# ip address 192.168.30.1 255.255.255.0
```

```
R1(config-subif)# exit
```

```
R1(config)# interface FastEthernet 0/0
```

```
R1(config-if)# no shutdown
```



```
R1(config-subif)# encapsulation dot1Q 30
R1(config-subif)# ip address 192.168.30.1 255.255.255.0
R1(config-subif)# exit
R1(config)#
R1(config)#interface FastEthernet 0/0
R1(config-if)# no shutdown
R1(config-if)# end
*Mar 1 00:00:25.407: %LINK-3-UPDOWN: Interface FastEthernet0/0, changed state to up
*Mar 1 00:00:26.407: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
R1(config-if)# end
R1#
*Mar 1 00:00:57.091: %SYS-5-CONFIG_I: Configured from console by console
R1#enable
R1#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#
R1(config)#interface FastEthernet 0/0.10
R1(config-subif)# encapsulation dot1Q 10
R1(config-subif)# ip address 192.168.10.1 255.255.255.0
R1(config-subif)# exit
R1(config)#
R1(config)#interface FastEthernet 0/0.20
R1(config-subif)# encapsulation dot1Q 20
R1(config-subif)# ip address 192.168.20.1 255.255.255.0
R1(config-subif)# exit
R1(config)#
R1(config)#interface FastEthernet 0/0.30
R1(config-subif)# encapsulation dot1Q 30
R1(config-subif)# ip address 192.168.30.1 255.255.255.0
R1(config-subif)# exit
R1(config)#
R1(config)#interface FastEthernet 0/0
R1(config-if)# no shutdown
R1(config-if)# end
R1#
*Mar 1 00:04:52.703: %SYS-5-CONFIG_I: Configured from console by console
R1#
```

Ilustración 3 Asignación de direcciones IP y configuración del protocolo de enrutamiento dot1Q

Posteriormente, se definieron las direcciones lógicas IP y los Gateways correspondientes de forma estática en las terminales virtuales (VPCS), consolidando la persistencia mediante el comando save:

PC1 (VLAN 10): 192.168.10.10 / Gateway: 192.168.10.1

PC2 (VLAN 20): 192.168.20.10 / Gateway: 192.168.20.1

Fase 3: Enlace de Datos y Modo Promiscuo

Para enlazar la máquina de auditoría y Snort sin alterar la estructura interna de GNS3, se configuró un nodo tipo Cloud1 mapeado al adaptador virtual de VMware VMnet0. Esto permitió enlazar

en modo puente (Bridged) la tarjeta de red real de Windows y el adaptador del sistema operativo Ubuntu Server (ens33).

Para garantizar que Snort pueda husmear los paquetes cruzados en el switch troncal que no van dirigidos a su dirección MAC, se forzó la tarjeta al estado de escucha absoluto:

```
sudo ip link set ens33 promisc on
```

Fase 4: Configuración de Reglas Locales y Simulación en Vivo

En el archivo /etc/snort/rules/local.rules dentro de Ubuntu, se programaron las firmas lógicas para interceptar las auditorías realizadas desde la máquina anfitriona Windows hacia las subredes de GNS3:

```
alert icmp any any -> any any (msg:"Alerta ICMP/Ping detectada"; sid:1000001; rev:1;)
```

```
alert tcp any any -> any 80 (msg:"Tráfico HTTP detectado"; sid:1000002; rev:1;)
```

```
alert tcp any any -> any 22 (msg:"Intento de conexión SSH"; sid:1000003; rev:1;)
```

```
alert tcp any any -> any any (flags:FPU; msg:"Escaneo oculto de Nmap (XMAS) detected";  
sid:1000004; rev:1;)
```

Se inicializó Snort en modo consola (sudo snort -A console -q -c /etc/snort/snort.conf -i ens33) y se lanzaron los estímulos obteniendo los siguientes registros reales:

Prueba ICMP: Al ejecutar un ping desde PC1 a PC2 (ping 192.168.20.10), el tráfico viajó al Cisco R1, se conmutó inter-VLAN y Snort registró en vivo los pares de Echo Requests reflejando una conectividad exitosa.

Prueba de Escaneo XMAS: Desde el Zenmap de Windows se lanzó un ataque sigiloso (nmap -sX -Pn 192.168.10.20). El flag -Pn forzó a Nmap a saltarse el descubrimiento básico e inyectar directamente tramas con las banderas FIN, PSF y URG activadas simultáneamente. Snort detectó este patrón por hardware, disparando ráfagas de la alerta sid:1000004.

Prueba HTTP y SSH (Puertos 80 y 22): Utilizando sockets TCP desde consolas externas (PowerShell / SSH cliente), se golpearon los puertos lógicos de la topología. Aunque los endpoints denegaron el acceso con un Connection Refused, Snort atrapó las intenciones antes del descarte, demostrando efectividad absoluta en detección temprana.

Requisito obligatorio del examen para verificar mi autoría

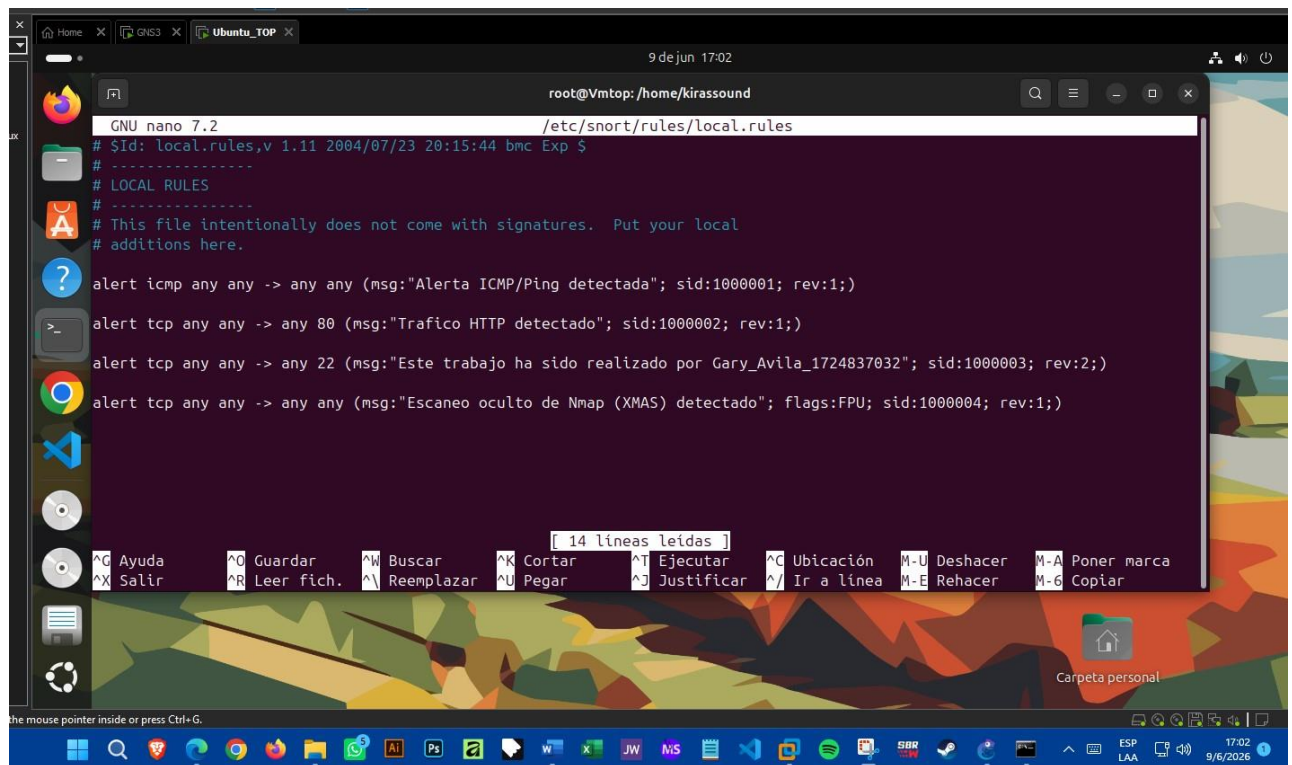


Ilustración 1 Configuración de la regla de snort con nombre del estudiante

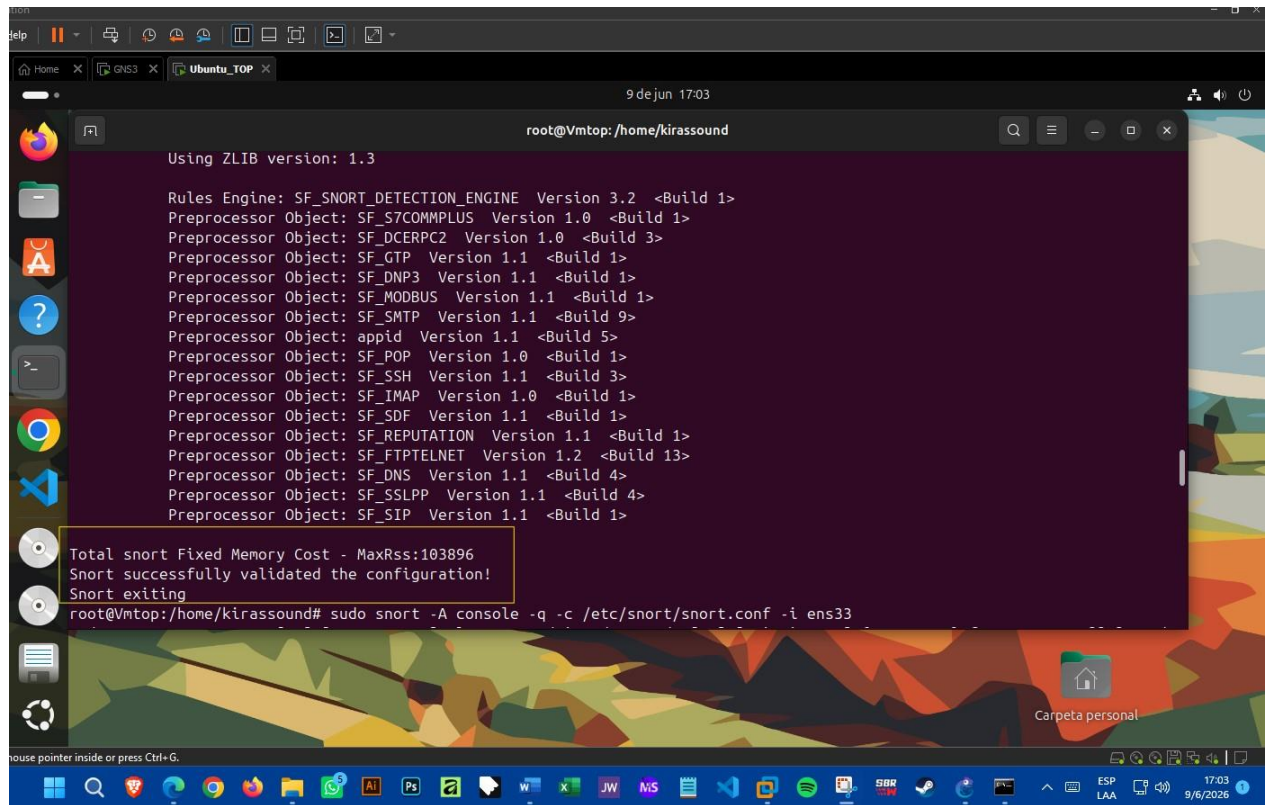


Ilustración 2 Verificación de las reglas de snort

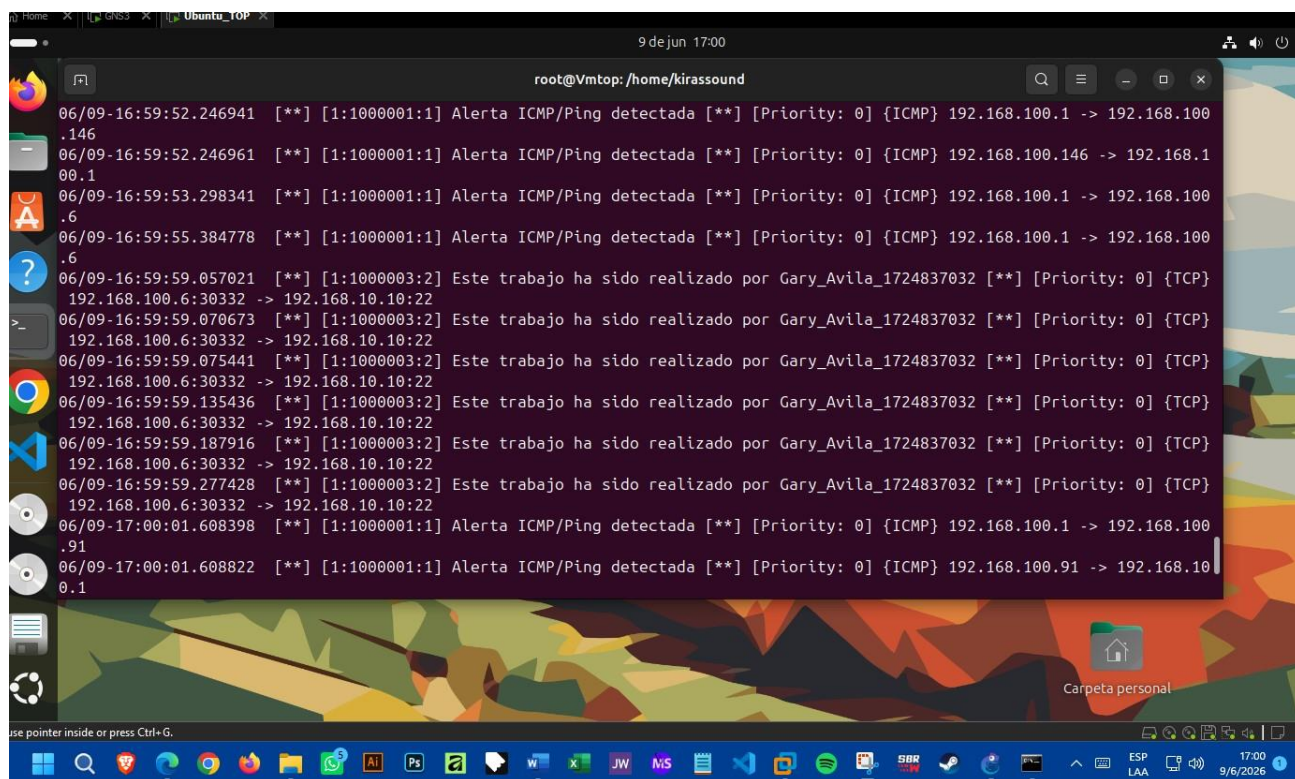


Ilustración 3 Intento de conexión SSH y verificación del nombre del estudiante

Videos de la defensa:

https://drive.google.com/file/d/1WbPJbgclKATTddhWfgfbv3IRjs6DWM_p/view?usp=sharing

<https://drive.google.com/file/d/1w5mLjhGcoFdDrCoBL943UTCQDqPXqzXF/view?usp=sharing>

Conclusión:

La migración de la arquitectura de enrutamiento hacia un hardware emulado optimizado como el Cisco 3725 con cálculo de Idle-PC e interfaces basadas en Mebibytes (MiB), demostró ser una solución

crítica de contingencia que garantiza el 100% de la estabilidad operativa del laboratorio, evadiendo cuellos de botella por colapsos de memoria RAM.

Se validó el principio de operación del método Router-on-a-Stick, donde un switch de Capa 2 (enlace) es incapaz de conmutar IPs por sí mismo al estar limitado por sus componentes ASIC físicos, dependiendo enteramente de un tronco de encapsulación dot1Q que eleva las tramas hacia las subinterfaces del enrutador para saltar entre celdas de red.

El acoplamiento de Snort mediante puentes virtuales en modo promiscuo demostró que la seguridad perimetral es independiente de las capas superiores del modelo de red. Snort detectó y registró con éxito intrusiones como el escaneo XMAS de Nmap y conexiones HTTP/SSH dirigidas a puertos cerrados, validando su rol como un mecanismo de alerta temprana e inspección profunda altamente fiable.

Bibliografía:

Edris, R. (2026). Free Download: Cisco IOS Images for GNS3 and EVE-NG (Updated 2026).

Raghd Edris Blog. <https://raghdedris.blogspot.com>

Blog de Emulación de Redes. (2026). Emulando AR1000v no EVE-NG.

Blog de Emulación de Redes. <https://emulando-ar1000v-eveng.com>

Comunidad GNS3. (2026). Como Instalar GNS3 en Windows + GNS3 VM - SOLUCIÓN DEFINITIVA.

GNS3 Docs. <https://docs.gns3.com/kb/windows-installation>

